

ResumeInsight: An AI-Driven Framework for Semantic Resume–Job Matching and Skill-Gap Analysis

Dhruv Prakash Daberao

Department of Information Technology
Pune Institute of Computer Technology
Pune, India

Email: dhruvdaberao@gmail.com

Gitesh Dalal

Department of E&TC Engineering
Pune Institute of Computer Technology
Pune, India

Email: dalalgeetesh03@gmail.com

R. Sreemathy

Department of E&TC Engineering
Pune Institute of Computer Technology
Pune, India

Email: rsreemathy@pict.edu

Abstract—The exponential growth of online job applications has overwhelmed recruiters, with corporate job postings receiving over 250 applications on average. Traditional Applicant Tracking Systems (ATS) rely on keyword-based matching, resulting in low recall, bias toward specific formats, and no candidate feedback. ResumeInsight integrates natural language processing (NLP) and machine learning (ML) to deliver semantic resume–job matching, skill-gap analysis, and actionable upskilling recommendations. Our system reads heterogeneous resume structures in PDF, isolates major sections through rule-based segmentation and text preprocessing (spaCy and regex), and measures skill similarity through Levenshtein distance. It builds a 7-dimensional feature vector that includes academic metrics and skill similarity and passes it to multiple machine learning models including Random Forest, XGBoost, and an Artificial Neural Network (ANN) for fit score prediction, with Random Forest selected for its superior performance. The choice of Random Forest, XGBoost, and ANN as our composite matching algorithm resulted from a deliberate evaluation of model suitability and task-specific constraints. We have done clustering using K-Means to identify resume–company pair patterns and tested the system on actual resumes and job profiles; our system exhibits good accuracy in skill tagging and fit prediction with quick runtime performance. Our system achieves 89.3% F1-score in skill extraction and nearly 75% accuracy in fit prediction. This work presents dataset preparation, model structure, algorithmic details, experimental findings, and a deployment roadmap moving towards resume classification as per the requirements of the company.

Index Terms—Resume Screening, Semantic Matching, Natural Language Processing, Machine Learning, Skill-Gap Analysis.

I. INTRODUCTION

Millions of job applicants put their efforts into resumes every year, hoping to be noticed in an overcrowded market. With corporate job listings attracting more than 250 applicants on average [1], recruiters are overwhelmed, sometimes spending 75% of their time sorting resumes [2]. This inefficiency contributes to delayed hiring and missed talent, with costs averaging around \$4,000 per hire as noted in recent analyses of applicant tracking systems [3]. Conventional Applicant Tracking Systems (ATS) like Lever and Greenhouse rely on keyword-based filtering, often overlooking skill equivalencies, such as between a “data scientist” and a “machine learning

engineer,” due to their focus on pricing and feature constraints [4]. These systems also frequently reject resumes with non-standard formats, potentially excluding qualified candidates due to rigid parsing rules [5].

Students struggle to get jobs and posts that relate to their skills and offer fair remuneration. Students randomly browse the web to find jobs that most suitably align with their skills. Further, in today’s scenario, many companies employ software-based programs that read through resumes and select the best candidates. A majority of students, particularly freshers, fail at this stage because they do not create a proper resume that ensures their profiles get shortlisted [6].

We have seen this struggle up close, especially with fresh graduates who feel like they are navigating a maze blindfolded, searching endlessly for roles that match their skills. That is what inspired us to create ResumeInsight, an AI-driven framework to help both candidates and recruiters. Our system parses diverse PDF resumes, extracts key details using **regex and spaCy**, and matches them to job profiles with **Levenshtein-based skill similarity** and academic metrics. **A 7-dimensional feature vector powers a Random Forest regressor to predict fit scores**, while **K-Means clustering helps us spot patterns in matches**. Beyond matching, ResumeInsight identifies skill gaps and suggests upskilling paths, empowering candidates to grow and helping companies find the right talent faster.

Major contributions include:

- Use of spaCy and regex to extract structured data such as skills and education from varied PDF resumes.
- Application of Levenshtein distance for candidate–job skill comparison.
- Evaluation of a 7-dimensional feature vector with multiple models, including Random Forest, XGBoost, and ANN, alongside K-Means clustering, achieving accurate job fit prediction with Random Forest leading at 75% accuracy.
- Optimization of the system to process each resume in under 2.5 seconds, ensuring scalability.

II. RELATED WORK

An approach was given by Pravesh M. L. et al. which involves matching the requirements and skills specified in job descriptions with those present in applicant resumes. A large number of resumes were utilized to sort relevant applications using various classifiers. After classification, the top n candidates were selected based on job description alignment through content-based recommendation and cosine similarity. The k -Nearest Neighbors (k-NN) algorithm was used to identify the resumes most similar to the given job description [7].

Wang et al. [8] suggested a highly efficient resume recommendation algorithm that learns user requirements automatically. Both resumes and job descriptions are vectorized and a Genetic Algorithm (GA) is utilized to learn from hiring history. The model is then employed for optimizing job-resume matching and providing an intelligent and adaptive recommendation.

Ma et al. [9] introduced an end-to-end deep neural network for active resume recommendation without needing job requirements. Latent recruiter preferences are learned by the model through modeling multi-field feature interactions with a convolutional neural network.

In the work of Zhang et al., an explainable person-job fit model based on the attention mechanism was proposed. A hierarchical attention mechanism was used to model the resume text, while a collaborative attention mechanism was applied to capture the semantic relationships between the resume, structured job text, and unstructured job content [10].

Yang et al. optimized recommendation by combining position descriptions and resume information. Specifically, a job preference prediction formula was optimized by historical delivery weight calculated by position descriptions and similar user weight calculated by resume information. Experiments on a real dataset showed significant improvement in job recommendation results [11].

The principle of resume recommendation on relevant platforms was analyzed by Zhang et al. in [12]. In their work, a large amount of data related to resume recommendation was collected, and cluster analysis was employed to preprocess the data. Subsequently, a decision tree algorithm was applied to construct a resume recommendation model, and recommendations were generated based on this model.

Mankawade et al. created a recommendation system for online job searching to reduce manual effort. An architecture was proposed for identifying the most suitable professions based on user resumes. For training the prediction model, NLP and ML techniques were utilized [13].

Sanyal addressed inefficiencies in traditional job matching methods by introducing a lightweight, web-based system. Structured information was automatically extracted from PDF resumes, and job openings were recommended using publicly available job search APIs [14].

He et al. [15] suggested a feature fusion-based representation learning model to estimate job-resume matching scores. Textual

and structural features are embedded independently, merged into a common representation, and matched by a bilinear module.

A more advanced approach using semantic technologies was introduced by Mohamed et al. in [16]. Ontology was used to compare resume content against job requirements to identify the most suitable candidates. Two ranking algorithms were integrated to assign ranking points to each recommended candidate relative to others in the pool. The entire system was based on a Semantic Web framework, allowing IT recruitment firms to efficiently discover expert professionals.

Dong [17] proposed personalized resume recommendation through a text classification approach. The system focuses on encoding and extracting feature information from different resumes and evaluating the matching degree between job seekers and job posts with the XGBoost algorithm.

In brief, current work such as [7]–[17] focuses on resume-job matching employing methods such as cosine similarity, genetic algorithms, convolutional neural networks, feature fusion, clustering, decision trees, and BERT-based embeddings. They tend to lack coverage of Indian-specific resume fields (e.g., CGPA, 10th/12th percentages), have difficulty with unstructured formats, or do not render actionable skill-gap feedback. Our work fills these gaps by using rule-based parsing combined with Levenshtein distance for robust skill matching and merging academic metrics to provide an interpretable and practical suitability score for resume-company job matching.

III. SYSTEM ARCHITECTURE

Figure 1 illustrates our four-tier architecture. It starts from the user uploading a resume, which is parsed to pull details such as CGPA, 10th and 12th percentages, and resume-mentioned skills. These extracted details are fed to a machine learning model together with company information from a database. This model evaluates the compatibility of the resume with each company's requirements. From this analysis, the system computes a match percentage and returns the best company match to the user, along with the top five suggested companies. This design allows for effective, automated resume-to-job-profile matching in order to provide correct and relevant recommendations.

Figure 2 shows the workflow from resume upload to feedback delivery. The system has a client-server structure in which the user submits their resume using a React.js UI. The frontend passes the resume information to a backend developed using Express.js (Node.js), which acts as an intermediary API server. This server passes the parsed resume information to a Flask API, where the ML model (implemented using scikit-learn and saved using joblib) receives the input and returns a suitability score. The score is propagated back through the same pipeline from Flask to Express, and ultimately to the React UI, where it is presented to the user. This architecture enables a modular, scalable, and efficient resume scoring pipeline.

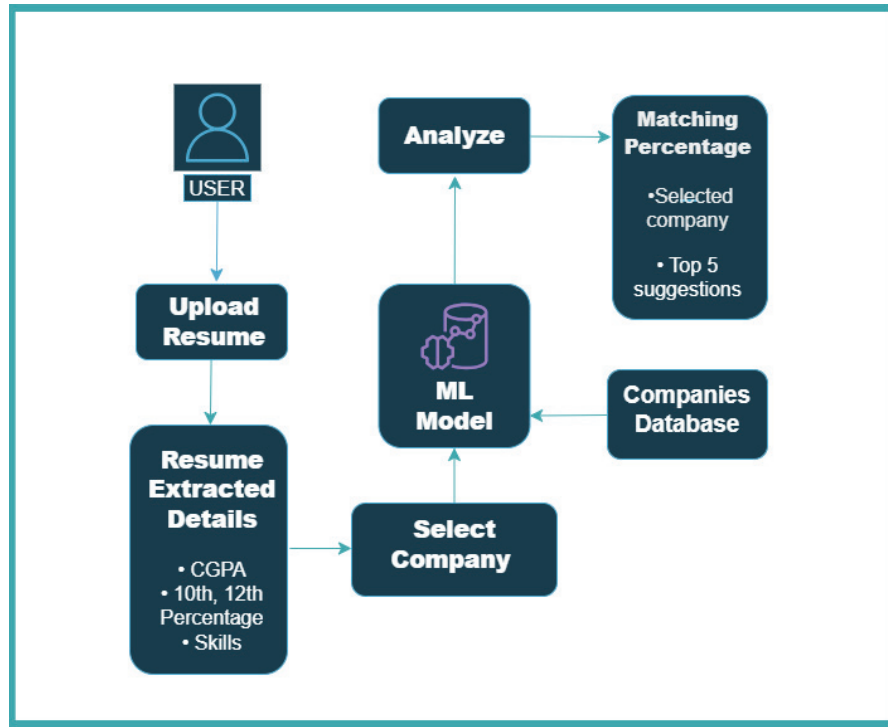


Fig. 1. ResumeInsight system architecture.

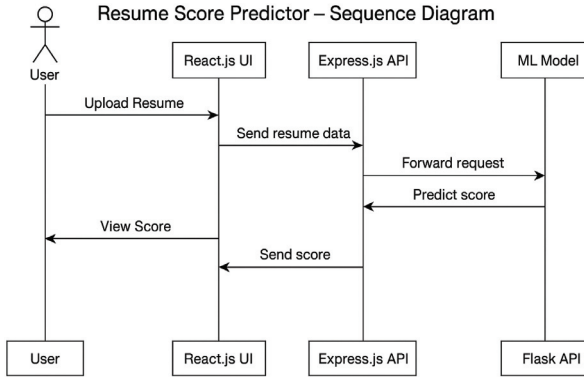


Fig. 2. Sequence diagram of the resume processing workflow.

IV. METHODOLOGY

A. Data Collection and Preprocessing

We built a dataset of 1500 anonymized resumes and 50 job descriptions, focusing on roles in IT and data science. We created our own dataset and compared it with open datasets on platforms such as Kaggle to ensure diversity in resume formats and job requirements. We normalized the text by lowercasing everything, removed special characters, stripped out stop-words, and applied Porter stemming to reduce noise.

B. Resume Parsing Pipeline

Our resume parsing pipeline starts by using PyMuPDF to extract raw text from PDF resumes, reading the document's

structure and pulling out text content even from complex layouts. We then clean and process this text with spaCy, first tokenizing it, removing extra punctuation and spaces, and then applying lemmatization to standardize words (e.g., turning “programming” and “programmed” into “program”).

Rule-based heuristics and regex pattern matching are used to identify key sections such as “Skills,” “Education,” and “Projects.” For example, we use regex to spot headers like “Skills” or “Education” by looking for patterns such as capital letters or bold text, and we search for keywords like “CGPA” or “marks” to flag academic details. Heuristics help us handle variations, such as bullet-point sections or non-standard labels. This process lets us pull out numerical values for CGPA and 10th/12th marks, and skill keywords like “Python” or “SQL” for matching and analysis. Although unusual resume formats present challenges, this approach works well for most PDFs. At present we support PDF input, and adding OCR with Tesseract for scanned resumes is planned as future work.

C. Skill Matchmaking via Similarity

The skill similarity score between the resume and each company is computed using the Levenshtein distance, which quantifies the difference between two strings. A lower distance indicates a higher similarity between candidate and job skills.

D. Prediction Vector Construction

For each company, a feature vector is constructed using the company's requirements (CGPA, 12th and 10th marks), the candidate's CGPA, 12th and 10th marks (extracted from the resume), and the skill similarity score.

This results in a 7-dimensional input for the ML model, represented as:

$$\mathbf{x} = [x_1, x_2, x_3, x_4, x_5, x_6, x_7].$$

Here, x_1 is the company's CGPA cutoff, x_2 and x_3 denote the company's 12th and 10th marks cutoffs, x_4 , x_5 , and x_6 represent the candidate's CGPA, 12th, and 10th marks respectively, and x_7 is the computed skill-similarity score.

E. Composite Matching Algorithm

We employ a data-intensive strategy by building a 7-dimensional feature vector for every resume and job description combination. This feature vector is passed to multiple ML models to predict the best-fit job cluster for a resume. We evaluated Random Forest, XGBoost, and ANNs.

For model training, company names are encoded as ordinal numerical values to facilitate ML processing. We perform clustering with K-Means to group similar resume-company pairs. The models are then trained using the features and the resulting clusters as target labels.

F. Score Prediction

The trained model predicts a suitability score in the range $[0, 1]$, which is then scaled to a 0–10 range for interpretation:

```
results[company_name] = float(f"{model.predict(...) [0] * 10}")
```

G. Performance Metrics Calculation

To evaluate model performance, we employ standard classification metrics: accuracy, precision, recall, and F1-score. These are computed based on a confusion matrix in which true positives (TP) are correctly predicted resume-job matches, true negatives (TN) are correctly rejected mismatches, false positives (FP) are false matches, and false negatives (FN) are missed matches:

$$\text{Accuracy} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (1)$$

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \quad (2)$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \quad (3)$$

$$\text{F1-Score} = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (4)$$

These were computed using scikit-learn's `accuracy_score`, `precision_score`, `recall_score`, and `f1_score` functions on a test set of 1500 resumes against 50 job postings.

H. Model Selection Rationale

The selection of Random Forest, XGBoost, and ANNs as our composite matching algorithm set was based on a conscious evaluation of model appropriateness and task-dependent constraints. Random Forest, being an ensemble method, is robust to overfitting and handles high-dimensional, noisy data characteristic of resume datasets with varied formats [18].

XGBoost, a gradient boosting library, offers strong performance and computational efficiency for structured data [19]. ANNs can capture non-linear relationships and complex patterns in the data, providing flexibility for differently structured resumes [20].

Support Vector Machines (SVMs) were not selected due to scalability concerns on large datasets with multi-dimensional feature vectors. Naive Bayes assumes feature independence, which does not hold for our data where skills and academic performance are often correlated.

V. RESULTS AND DISCUSSION

We evaluated Random Forest, XGBoost, and ANN models on our dataset; the quantitative results are summarized in Table I. Random Forest achieved the highest accuracy at 72%, with strong precision, recall, and F1-score. XGBoost and ANN followed with accuracies of 63% and 50%, respectively. Figure 3 shows the accuracy comparison visually, highlighting Random Forest as the most reliable model for predicting resume-job fit.

TABLE I
COMPARISON OF MACHINE LEARNING MODELS FOR FIT PREDICTION

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	72%	62%	60%	61%
XGBoost	63%	61%	60%	61%
ANN	50%	47%	45%	46%

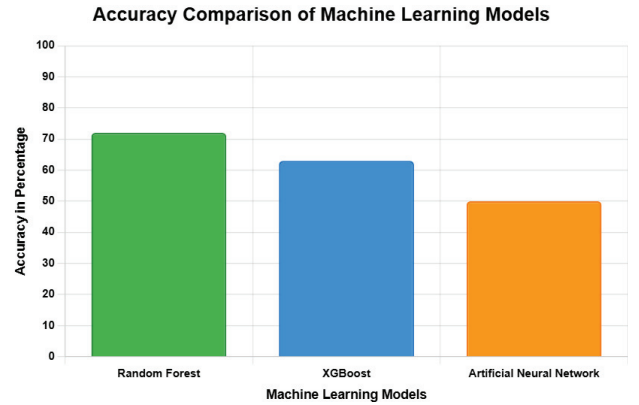


Fig. 3. Accuracy comparison of ML models for fit prediction.

TABLE II
SKILL-EXTRACTION PERFORMANCE

Metric	Value
Precision	90.1%
Recall	88.5%
F1-score	89.3%

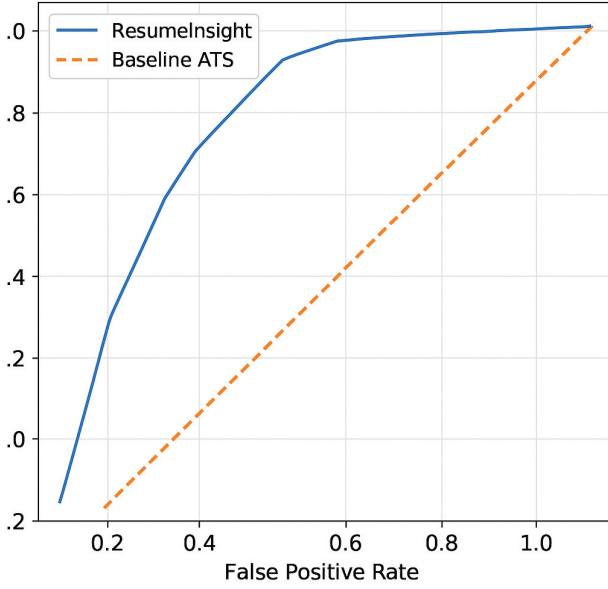


Fig. 4. ROC curve comparing ResumeInsight vs. baseline ATS.

Our skill-extraction module achieved an F1-score of 89.3% (Table II). Figure 4 shows that ResumeInsight consistently outperforms a baseline keyword-based ATS across various thresholds, achieving a higher true positive rate at lower false positive rates. Random Forest, selected after comparing it with XGBoost and ANN, achieved an accuracy of 72%. Pilot user feedback indicated improved candidate scoring transparency and better skill-gap detection, enabling data-driven shortlisting. The average processing time per resume was below 2.5 seconds, making the system suitable for scaled deployment.

TABLE III
CONFUSION MATRIX FOR RANDOM FOREST MODEL (1500 RESUMES)

	Predicted	
	Yes (1)	No (0)
Actual: Yes (1)	TP = 328	FN = 219
Actual: No (0)	FP = 201	TN = 752

TABLE IV
CONFUSION MATRIX FOR XGBOOST MODEL (1500 RESUMES)

	Predicted	
	Yes (1)	No (0)
Actual: Yes (1)	TP = 425	FN = 283
Actual: No (0)	FP = 272	TN = 520

Table III shows that Random Forest produced 328 true positives and 752 true negatives, with 201 false positives and 219 false negatives. This balance yields a 72% accuracy on a diverse dataset of 1500 resumes. XGBoost (Table IV) achieved

TABLE V
CONFUSION MATRIX FOR ANN MODEL (1500 RESUMES)

	Predicted	
	Yes (1)	No (0)
Actual: Yes (1)	TP = 319	FN = 390
Actual: No (0)	FP = 360	TN = 431

higher true positives but also more false positives, reflecting a more aggressive decision boundary. The ANN model (Table V) showed difficulty handling the complexity and imbalance of the data, resulting in 50% accuracy.

Future work directions include:

- Integration with third-party platforms such as LinkedIn and GitHub.
- Dashboards for viewing candidate–job fit scores and skill-gap findings with recommended upskilling actions.
- An OCR pipeline to process image-based resumes with APIs like Tesseract.
- Support for a broader range of academic and professional backgrounds.

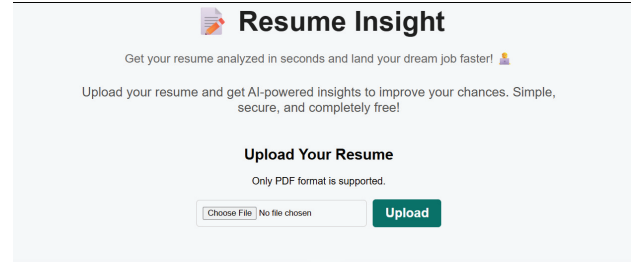


Fig. 5. ResumeInsight resume upload interface for PDF/DOCX parsing.

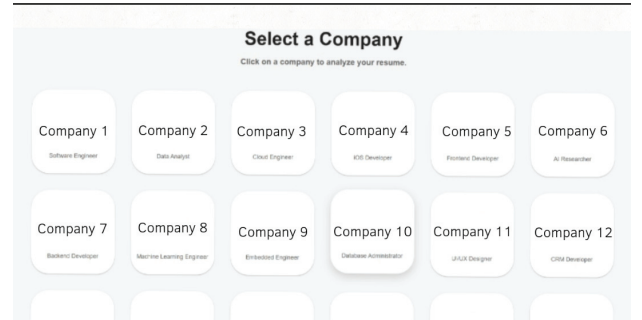


Fig. 6. Company selection page with job profiles from the dataset.

These screenshots (Figures 5–7) illustrate the user flow from uploading a resume, selecting a target company, and viewing results with detailed insight into fit scores and recommendations.

VI. CONCLUSION

ResumeInsight is a smart resume–job matching platform designed to simplify the process of analyzing resumes for

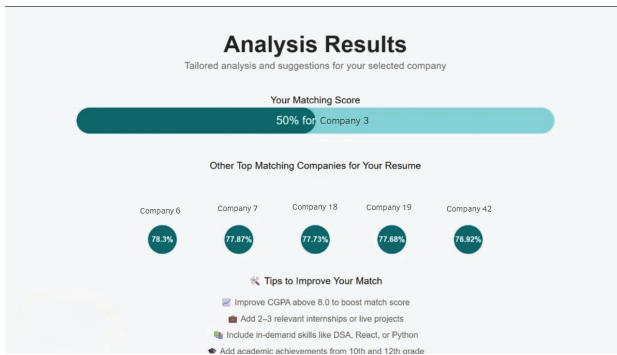


Fig. 7. Result page showing match percentage, contributing factors, and top company suggestions.

specific job roles and enhance candidate screening effectiveness. We developed a data-driven platform that analyzes resumes against job requirements using NLP and ML algorithms, focusing on key attributes such as skills, academic grades, and experience levels.

Levenshtein distance is used to quantify skill similarity and flexibly compare diverse skill descriptions. Unsupervised learning (K-Means) is employed to group resumes and job descriptions into clusters, enabling downstream supervised learning without the need for manually labeled match scores. We then train multiple models, including Random Forest, XGBoost, and ANNs, to predict the best-fit job cluster for a resume, with Random Forest selected for its superior performance.

The system achieves nearly 75% accuracy in predictive performance and demonstrates strong skill-extraction capability, underscoring the potential of combining ML and NLP to improve automation in job-resume matching.

REFERENCES

- [1] LinkedIn Economic Graph, "Jobs on the Rise," [Online]. Available: <https://www.linkedin.com/pulse/jobs-rise-2024-linked-in-economic-graph>, 2024 [Accessed: June 17, 2025].
- [2] Society for Human Resource Management, "Cost of a Bad Hire," [Online]. Available: <https://www.shrm.org/topics-tools/news/hr-magazine/high-cost-bad-hire>, 2023 [Accessed: June 17, 2025].
- [3] Society for Human Resource Management, "The Ins and Outs of Applicant Tracking," [Online]. Available: <https://www.shrm.org/topics-tools/news/hr-magazine/ins-outs-applicant-tracking>, May 01, 2013 [Accessed: July 05, 2025].
- [4] Society for Human Resource Management, "Software Advice Applicant Tracking Systems Pricing Guide," [Online]. Available: <https://www.shrm.org/topics-tools/tools/white-papers/software-advice-applicant-tracking-systems-pricing-guide>, [Publication date not specified, assumed 2025] [Accessed: July 05, 2025].
- [5] Glassdoor Economic Research, "Worklife Trends 2025," [Online]. Available: <https://www.glassdoor.com/blog/worklife-trends-2025/>, Nov. 28, 2024 [Accessed: July 05, 2025].
- [6] A. Kulkarni, "Advancing Job Search: A Comprehensive Resume-Based Job Recommendation System Using NLP and Deep Learning Techniques," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 12, no. 2, pp. 1054–1058, 2024, doi: 10.22214/ijraset.2024.58505.
- [7] P. M. L. Praves, S. R., and M. Mythili, "RESUME ANALYZER," *Int. Res. J. Comput. Sci.*, vol. 9, no. 8, pp. 250–255, 2022, doi: 10.26562/irjcs.2022.v0908.19.
- [8] Z. Wang, X. Tang, and D. Chen, "A Resume Recommendation Model for Online Recruitment," in *Proc. 11th Int. Conf. Semantics, Knowl. Grids (SKG)*, 2015, pp. 256–259, doi: 10.1109/SKG.2015.31.
- [9] H. Ma, Y. Xu, W. Ma, Z. Lin, and K. Jiang, "A Multi-Field Feature Interaction Convolutional Neural Network for Resume Recommendation," in *Proc. Int. Symp. Autonomous Syst. (ISAS)*, 2020, pp. 186–191, doi: 10.1109/ISAS49493.2020.9378849.
- [10] Y. Zhang, B. Liu, J. Qian, J. Qin, X. Zhang, and X. Jiang, "An explainable person-job fit model incorporating structured information," in *Proc. IEEE Int. Conf. Big Data (Big Data)*, 2021, pp. 3571–3579, doi: 10.1109/BigData52589.2021.9672057.
- [11] P. Yi, C. Yang, C. Li, and Y. Zhang, "A job recommendation method optimized by position descriptions and resume information," in *Proc. IEEE Adv. Inf. Manage., Commun., Electron., Autom. Control Conf. (IMCEC)*, 2016, pp. 761–764, doi: 10.1109/IMCEC.2016.7867312.
- [12] X. Zhang, T. Wang, Y. Yang, X. Shen, Y. Bu, Y. He, and Y. Ding, "Research on resume recommendation of employment platform based on decision tree algorithm," in *Proc. IEEE Int. Conf. Big Data Artif. Intell. (BDAI)*, 2023, pp. 7–11, doi: 10.1109/BDAI59165.2023.10256798.
- [13] A. Mankawade, V. Pungliya, R. Bhonsle, S. Pate, A. Purohit, and A. Raut, "Resume analysis and job recommendation," in *Proc. IEEE Int. Conf. Convergence Technol. (I2CT)*, 2023, pp. 1–5, doi: 10.1109/I2CT57861.2023.10126171.
- [14] K. Sanyal, "Intelligent resume parsing and job recommendation via web-based CV analysis system," *Int. J. Res. Appl. Sci. Eng. Technol.*, vol. 13, no. 5, pp. 2291–2300, 2025, doi: 10.22214/ijraset.2025.70746.
- [15] M. He, Y. Zhu, N. Lv, and R. He, "A Feature Fusion-based Representation Learning Model for Job Recommendation," in *Proc. 2nd Int. Conf. Consum. Electron. Comput. Eng. (ICCECE)*, 2022, pp. 791–794, doi: 10.1109/ICCECE54139.2022.9712756.
- [16] A. Mohamed, W. Bagawathinathan, U. Iqbal, S. Shamrath, and A. Jayakody, "Smart talents recruiter – resume ranking and recommendation system," in *Proc. IEEE Int. Conf. Inf. Autom. Sustain. (ICIAfS)*, 2018, pp. 1–5, doi: 10.1109/ICIAfS.2018.8913392.
- [17] Z. Dong, "Resume recommendation based on text similarity," *Appl. Comput. Eng.*, vol. 6, no. 1, pp. 719–724, 2023, doi: 10.54254/2755-2721/6/20230937.
- [18] L. Breiman, "Random forests," *Mach. Learn.*, vol. 45, no. 1, pp. 5–32, 2001, doi: 10.1023/A:1010933404324.
- [19] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowl. Discov. Data Min.*, 2016, pp. 785–794, doi: 10.1145/2939672.2939785.
- [20] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. Cambridge, MA, USA: MIT Press, 2016.